

Project Tricorder Subspace Protocol

Discovery, Push-to-Talk, and Event Frames over a Meshtastic Text Bearer

Internet-Draft: draft-tricorder-subspace-00

Intended status: Informational

Version analyzed: Project Tricorder 0.6.138 source tree

Date: 6 October 2026

Author: Jim Ames / N2NHU Labs

This document is an independent implementation specification. It is not an IETF document, is not endorsed by the IETF, and has no RFC number.

Abstract

This document specifies the SUBSPACE application protocol implemented by Project Tricorder 0.6.138 for exchanging presence, position, push-to-talk (PTT) speech, and field-detection events over a Meshtastic text-message bearer.

SUBSPACE deliberately does not modify Meshtastic firmware and does not attempt to turn LoRa into a conventional connected transport. The Tricorder treats the Meshtastic text service as an unreliable, asynchronous datagram path. Application semantics are therefore carried explicitly in printable text frames.

The protocol consists of four actively generated wire formats: AH presence beacons, POS position reports, PTT speech datagrams, and TF fragmented event frames. The implementation also recognizes legacy TC frames, but the 0.6.138 source tree does not generate them.

This specification describes not merely the wire grammar but the operational model surrounding it: Android BLE attachment to a Meshtastic node, protobuf encapsulation, transmission pacing, discovery timing, event classification and prioritization, fragmentation, reassembly, partial-delivery behavior, duplicate/self suppression, queue persistence, failure behavior, and security boundaries. The purpose is to make the protocol independently implementable and maintainable without requiring the reader to infer behavior from the Tricorder application.

1. Introduction

A Meshtastic node provides the radio and mesh forwarding system. Project Tricorder provides the instrument semantics.

The distinction is fundamental. The radio does not know that a payload represents an aircraft detection, a speech utterance, a location beacon, or a remote tricorder. SUBSPACE supplies that missing application layer.

The protocol was designed around a field instrument rather than a general-purpose messaging service. The design consequently favors:

- short printable frames;
- broadcast-oriented discovery;
- best-effort delivery;
- explicit application framing;

- bounded transmission rate;
- local persistence of the field-log evidence;
- graceful degradation when packets are lost;
- no application-level handshake for ordinary operation;
- speech and event delivery that can operate without a pre-existing session.

The resulting system is better understood as **store, offer, transmit, hear, classify, and speak** than as a socket connection.

2. Terminology and Conventions

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described by RFC 2119 and RFC 8174.

Tricorder

A Project Tricorder instrument running the application.

Node

The Meshtastic radio device attached to a Tricorder. In the reference implementation this is a SenseCAP T1000-E, but SUBSPACE is not intrinsically tied to that model.

Bearer

The Meshtastic text-message application transported over the node's radio network.

SUBSPACE

The Tricorder application protocol described here.

Logical event

One detection offered to SUBSPACE for transmission. A logical event may become one or several `TF` frames.

Frame

One printable ASCII line sent through the Meshtastic text bearer.

Peer

A remote Tricorder discovered through `AH`, `POS`, or `PTT` traffic.

3. System Architecture

The reference implementation has three distinct communication layers.

3.1 Tricorder application layer

Detection engines and field-log operations call the SUBSPACE sidecar with:

- an event identifier;

- a source kind;
- human-readable text;
- an optional latitude;
- an optional longitude.

The sidecar classifies the event, decides whether the category is enabled, queues it, applies priority, fragments it, and passes resulting text lines to the bearer.

The field log remains the evidence book. SUBSPACE maintains only a transmission sidecar and recent transmission history.

3.2 Android/Meshtastic bearer layer

`MeshLink` attaches to the Meshtastic node over Bluetooth Low Energy.

The reference implementation:

1. scans for a Meshtastic-compatible BLE device;
2. recognizes the Meshtastic service UUID or a matching device name;
3. connects with GATT;
4. requests a 512-byte BLE MTU;
5. discovers the Meshtastic service and its radio characteristics;
6. constructs `TEXT_MESSAGE_APP` protobuf packets;
7. writes packets to the node;
8. reads packets from the node;
9. extracts text messages;
10. passes application text to the Tricorder protocol handler.

The Tricorder therefore does **not** implement LoRa modulation, LoRa MAC behavior, or Meshtastic mesh routing. Those remain below SUBSPACE.

3.3 SUBSPACE protocol layer

The protocol handler recognizes the application frame prefix:

- `'AH|'`
- `'POS|'`
- `'PTT|'`
- `'TF|'`
- legacy `'TC|'`

Everything above this boundary is Tricorder application semantics.

4. Transport Model

SUBSPACE uses the Meshtastic `TEXT_MESSAGE_APP` service as a text bearer.

The reference implementation constructs a Meshtastic packet with:

- application port `'TEXT_MESSAGE_APP'`;
- configurable destination, defaulting to broadcast `'ffffff'`;
- hop limit three in the local protobuf encoder;
- UTF-8 payload bytes.

The radio destination is therefore a bearer parameter, not an address encoded into the SUBSPACE frames.

4.1 Broadcast is the normal model

The default destination is broadcast. Discovery and event distribution are consequently one-to-many operations.

An implementation MAY configure a different Meshtastic destination, but the SUBSPACE protocol itself does not define unicast routing or peer addressing.

4.2 No connection state

SUBSPACE does not create a connection before transmitting.

There is:

- no SYN;
- no session identifier;
- no application-level ACK;
- no retransmission protocol;
- no stream;
- no negotiated MTU;
- no delivery receipt.

A receiver that hears a valid frame may act on it. A receiver that does not hear it has no protocol mechanism to request it again.

This is intentional.

4.3 Transmission pacing

The bearer maintains a transmit queue.

Only one queued text line is emitted at a time. After a send, the reference implementation waits approximately 2.5 seconds before allowing the next queued line to leave, with the interval configurable between 800 ms and 8 seconds.

This pacing is distinct from the SUBSPACE event token bucket.

The result is two layers of rate control:

11. ****SUBSPACE logical-event rate:**** one to five events per minute, operator-configurable.
12. ****Bearer line pacing:**** approximately 2.5 seconds between queued radio sends by default.

The discovery beacon is outside the event token bucket, but its `AH` and `POS` lines still pass through the bearer queue and pacing.

5. Wire Encoding

Every SUBSPACE frame is a single printable ASCII line.

The vertical bar (`|`) is the field delimiter.

The implementation replaces a literal `|` in event and speech text with `/`. Newline characters in event labels are replaced by spaces.

Implementations MUST NOT assume that a field can contain an unescaped `|`.

5.1 ABNF

The following grammar describes the actively generated protocol:

```

frame      = aloha / position / ptt / tf

aloha      = "AH|" name "|" tail "|" latitude "|" longitude "|" time
position   = "POS|" name "|" latitude "|" longitude
ptt        = "PTT|" name "|" [latitude "|" ] [longitude "|" ] text
tf         = "TF|" msgid "|" sequence "|" category "|" body

name       = 1*16(printable)
tail       = 1*16(HEXDIG)
msgid      = 1*4(HEXDIG)
sequence   = 1*1DIGIT "/" 1*1DIGIT
category   = 1*(ALPHA / DIGIT / "_" / "-")
latitude   = number / "na"
longitude  = number / "na"
time       = 2DIGIT ":" 2DIGIT
text       = *printable
body       = *printable
number     = ["-"] 1*DIGIT ["." 1*DIGIT]

```

The ABNF intentionally describes the interoperable envelope rather than pretending that the current Kotlin parser is a formal parser. The implementation is permissive in several places and performs field-level conversion after splitting.

5.2 Practical PTT variants

Two PTT forms exist in the 0.6.138 implementation.

The scheduled/utility beacon form is:

```
PTT|NAME|LAT|LON|aloha
```

A normal PTT row sent through `MeshLink.send(row)` is currently:

```
PTT|NAME|TEXT
```

The receiver accepts both forms.

This is an important implementation detail and is intentionally preserved here rather than silently normalizing the wire protocol.

6. Presence Discovery: AH

AH means ALOHA-style presence announcement.

Format:

```
AH|name|tail|lat|lon|hh:mm
```

Example:

```
AH|KIRK|7A3F|41.50321|-74.01022|09:42
```

6.1 Semantics

name is the Tricorder callsign, limited by the sender to 16 characters.

`tail` is normally the last four hexadecimal characters of the attached Meshtastic radio address. If no radio tail is available, the implementation falls back to the last four characters of the Tricorder GUID.

`lat` and `lon` are decimal degrees to five places.

`hh:mm` is the sender's local clock representation.

The receiver uses the name, tail, and coordinates. The current receiver does not parse the timestamp field into a peer timestamp.

6.2 Scheduling

When SUBSPACE is armed, the first ALOHA attempt occurs eight seconds later.

Thereafter the configured interval is one or three minutes.

An ALOHA transmission is suppressed while:

- the Meshtastic bearer is not linked; or
- no finite local GPS fix exists.

When either condition is absent, the scheduler retries after approximately 20 seconds.

6.3 No reply

The source tree contains an `alohaReply()` implementation that can construct an `|R` reply marker, but the reply is intentionally disabled. The active discovery path does not answer received `AH` frames.

This prevents discovery from becoming an ACK storm.

7. Position: POS

Format:

```
POS|name|lat|lon
```

Example:

```
POS|KIRK|41.50321|-74.01022
```

`POS` repeats the current position without the radio tail and time.

The reference implementation sends `AH` followed by `POS` during each discovery cycle.

`POS` is not spoken.

A receiver ignores a `POS` belonging to its own callsign and otherwise plots the peer.

8. Push-to-Talk: PTT

PTT carries recognized speech rather than a field-log detection.

The full form is:

```
PTT|name|lat|lon|text
```

The compact form used by the generic PTT sender is:

```
PTT|name|text
```

The receiver distinguishes the two by field count.

8.1 PTT is datagram speech

PTT does not establish a voice session.

Each received PTT is treated as a text utterance and passed to the Tricorder speech-output path.

The receiver renders the utterance conceptually as:

```
Beep beep beep. From NAME. TEXT
```

8.2 Incremental speech suppression

The receiver remembers the previous PTT source, text, and receive time.

If a newer utterance from the same source arrives within eight seconds and begins with the previous text while being longer, the current speech output is stopped and the longer text replaces it.

This provides a primitive application-level replacement mechanism for growing recognition results without requiring acknowledgements.

It is not TCP-style retransmission and MUST NOT be interpreted as reliable delivery.

8.3 PTT as discovery

The literal speech token:

```
aloha
```

is reserved by the current receiver as a beacon indication. Such a PTT is not spoken as ordinary speech.

A PTT containing coordinates can also establish or update a peer plot even when an AH has not yet been received.

9. Event Transmission: TF

TF means Tricorder Fragment.

Format:

```
TF|id|i/n|category|body
```

Example:

```
TF|A91C|1/2|flight|09:42 41.503,-74.010 aircraft detected...
TF|A91C|2/2|flight| tail N123AB
```

9.1 Logical event construction

A field-log row offered to SUBSPACE is first classified into one of the following categories:

```
weapon
civic
flight
human
vehicle
```

```

flora
motion
soundscape
silence

```

The current default transmission policy enables:

- `weapon`
- `civic`
- `flight`

and disables the remaining categories unless the operator enables them.

The policy is an operator setting, not a property of the wire protocol.

9.2 Priority

When several events are waiting, the reference implementation considers them in this order:

```

weapon
civic
flight
human
vehicle
flora
motion
soundscape
silence

```

Within a selected category, the oldest pending event is transmitted first.

This makes SUBSPACE a prioritized alert channel rather than a FIFO copy of the entire field log.

9.3 Event text

The first fragment begins with:

```
HH:mm LAT,LON LABEL
```

where coordinates are rounded to three decimal places. If no valid fix exists, the place field is `na`.

The label is capped at 480 characters before fragmentation.

A | in the label becomes /.

10. Fragmentation

The current source defines:

```
CHUNK = 140
```

characters.

The event body is divided into sequential chunks of at most 140 characters.

Each chunk is emitted as:

```
TF|ID|I/N|CATEGORY|CHUNK
```

The fragment count is encoded as `N`.

The sender currently permits up to eight fragments at the wire layer, although the normal 480-character label cap generally produces no more than four fragments.

10.1 Fragment identity

The sender obtains an eight-character uppercase prefix from its Tricorder GUID after removing hyphens, then places the first four characters into the TF identifier.

Therefore the effective current wire identifier is four hexadecimal characters.

This is not a globally unique message identifier.

The identifier is currently derived from the sender identity rather than generated uniquely for each logical event.

11. Reassembly

The receiver maintains a map of incomplete fragment sets.

The reassembly key is the four-character TF identifier.

For each set, the receiver stores:

- expected fragment count;
- category;
- an array of fragment bodies;
- whether the alert tone has already been emitted.

When all fragments are present, the receiver concatenates them in index order and speaks the logical event once.

11.1 Single-frame event

For:

```
TF|A91C|1/1|flight|text
```

the receiver speaks immediately.

11.2 Multi-frame event

For a multi-frame event, the first received fragment produces an alert tone and a UI note. The receiver then waits for the complete set.

The current timeout is 2.5 seconds.

If all fragments arrive before the timeout, the complete event is spoken.

If the timer expires first, the receiver:

13. removes the fragment buffer;
14. concatenates only the fragments received;
15. speaks the partial text if it is non-empty.

There is no negative acknowledgement and no retransmission request.

11.3 Important collision property

Because the current identifier is derived from the sender GUID rather than a per-message sequence, two overlapping multi-frame events from the same sender can use the same reassembly key.

The implementation therefore has a real protocol limitation:

The TF identifier identifies a sender-derived fragment namespace, not an individual logical message.

This is a known defect and should be corrected in a future revision.

12. Echo Prevention and Remote Semantics

SUBSPACE is designed to prevent a remote detection from immediately becoming a new local detection and being offered back to the mesh.

The application refuses to offer text beginning with:

Remote tricorder

or:

Local tricorder detected

to the SUBSPACE queue.

Remote events are spoken and written to the local `comlink` field-log stream, but they do not re-enter the outbound event pipeline through the normal `offerSubspace()` path.

This is an application-level echo break.

It is not a cryptographic loop-prevention mechanism.

13. Rate Control

SUBSPACE uses a token bucket for logical event transmission.

The operator selects one through five events per minute.

The bucket is initially populated with one token and refills continuously according to the selected rate, capped at the rate value.

A successful `pump()` operation:

16. checks that LoRa is enabled;
17. refills the bucket;
18. selects the highest-priority enabled category containing a pending event;
19. selects the oldest pending event in that category;
20. fragments it;
21. submits every fragment to the bearer queue;
22. removes the event from the pending queue;
23. records the event as blasted;
24. consumes one token.

The event rate therefore counts **logical detections**, not individual TF fragments.

A four-fragment event consumes one SUBSPACE event token but produces four bearer transmissions.

This distinction is operationally important.

14. Queue and Persistence

The SUBSPACE sidecar keeps a concurrent pending map and a small recent-send history.

The sidecar writes its state to:

```
<application files>/logs/subspace.json
```

The persisted array is capped at 400 records.

The field log remains authoritative evidence; `subspace.json` is operational transmission history.

Clearing the SUBSPACE queue removes pending entries and clears the bearer transmit queue. It does not constitute deletion of the field log.

15. Receive Path

The receive path is intentionally tolerant.

The Meshtastic protobuf decoder first extracts `TEXT_MESSAGE_APP` bodies.

A secondary byte-level text sniffer scans received bytes for:

```
PTT|  
POS|  
AH|  
TF|  
TC|
```

This fallback allows the application to recover recognizable text even when the surrounding protobuf payload is not decoded through the primary path.

Each resulting line is passed to the Tricorder application handler.

The handler dispatches by prefix:

```
AH -> peer discovery  
POS -> peer position  
PTT -> speech / beacon  
TF -> event reassembly  
TC -> legacy event handling
```

Unknown application objects may still enter the general Comlink path, but they are not part of the normative SUBSPACE wire set.

16. Bearer Failure and Recovery

The bearer is expected to fail.

Examples include:

- Bluetooth disabled;
- no BLE scanner;
- no matching Meshtastic node;
- GATT service unavailable;
- GATT disconnection;
- transient write failure;
- no GPS fix;
- queue buildup;
- packet loss on the LoRa network.

On BLE disconnection, the reference implementation marks the link down, closes the GATT object, and attempts to restart discovery after approximately four seconds while the feature remains enabled.

A pending SUBSPACE event remains pending until `pump()` can transmit it.

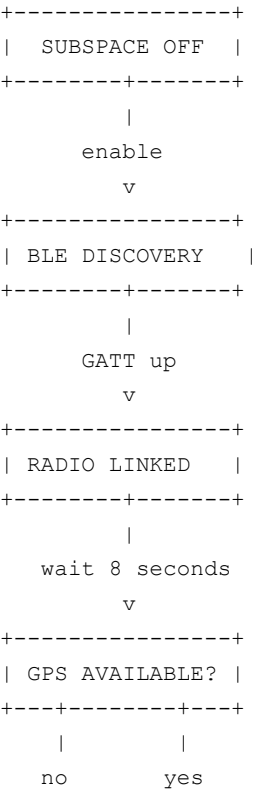
Once `pump()` submits a frame to the bearer, SUBSPACE considers that frame offered to the bearer. It does **not** know whether the LoRa network delivered it.

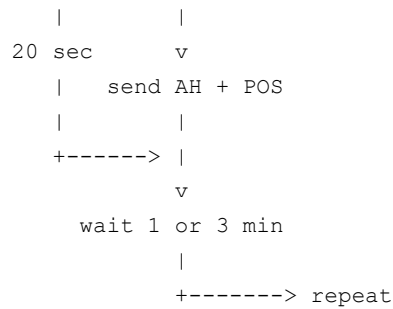
This is a crucial boundary:

SUBSPACE knows queued, submitted, and heard. It does not know delivered.

17. Discovery State Machine

The logical discovery state can be represented as:

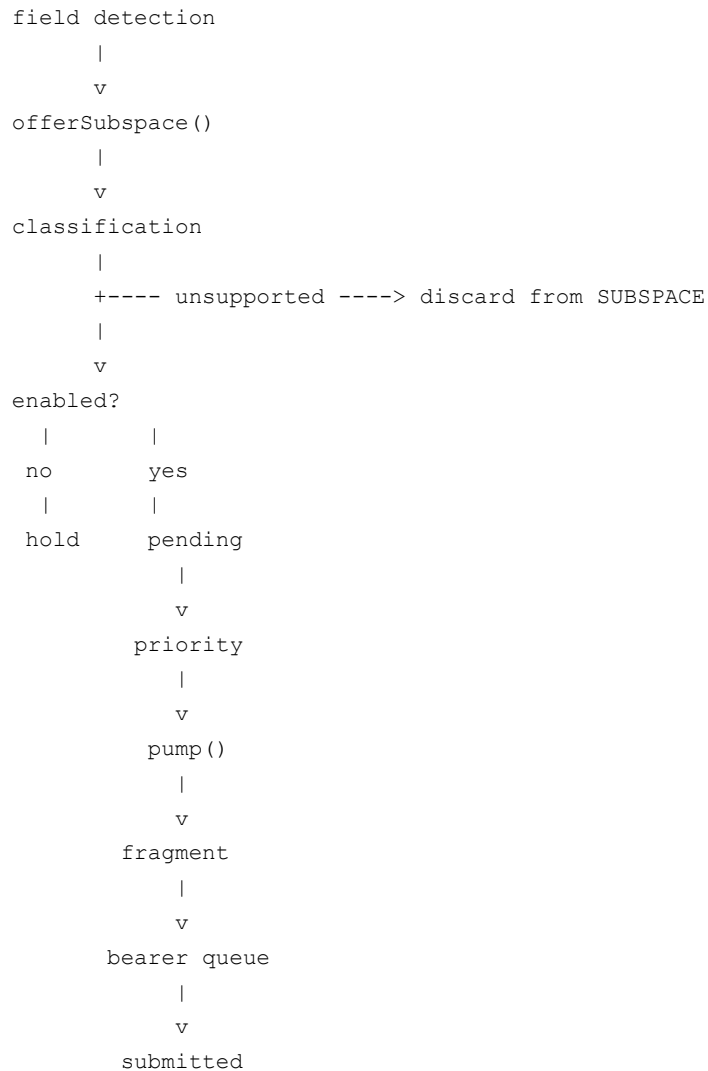




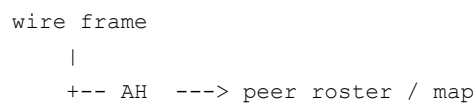
This state machine is local to the Tricorder. It is not a Meshtastic routing state machine.

18. Event State Machine

A logical detection follows:



The receiver follows:



```
|
+-- POS ---> peer position
|
+-- PTT ---> speech
|
+-- TF ---> reassembly
      |
      +-- complete ---> speak once
      |
      +-- timeout ---> speak partial
```

19. What SUBSPACE Is Not

SUBSPACE is not:

- a replacement for Meshtastic;
- a LoRa physical-layer protocol;
- an authenticated identity system;
- a reliable transport;
- a TCP-like stream;
- a voice codec;
- a guaranteed delivery system;
- a distributed database;
- a synchronized event log;
- a general-purpose mesh routing protocol.

Its job is narrower:

Give independent Tricorders a small common language for announcing themselves, talking, and sharing selected detections over an existing off-grid text bearer.

20. Security Considerations

The application protocol itself provides no authentication.

A callsign is an identifier, not proof of identity.

The four-character TF identifier is not a cryptographic message identifier.

A latitude/longitude pair is an asserted observation, not a verified location.

An implementer requiring stronger identity **SHOULD** use the security properties of the underlying Meshtastic channel and/or add authentication above the current wire grammar.

The reference implementation intentionally keeps identity and trust outside the callsign field.

Operators should treat remote detections as observations received over a best-effort radio channel, not as authoritative facts.

Location information is particularly sensitive. Deployments should use a private channel when location and civic/field observations are not intended for every compatible node in radio range.

21. Privacy Considerations

AH, POS, and coordinate-bearing PTT frames disclose location.

TF frames normally carry a rounded coordinate in the first fragment.

A deployment that enables SUBSPACE should therefore consider:

- who possesses the Meshtastic channel key;
- who can receive the radio traffic;
- whether the selected channel is appropriate for location sharing;
- whether event categories such as human detection should be transmitted.

The protocol has no privacy mode independent of the bearer.

22. Interoperability Requirements

A conforming receiver should:

25. accept `AH`, `POS`, `PTT`, and `TF`;
26. tolerate unknown fields where possible;
27. ignore its own discovery frames;
28. accept compact and coordinate-bearing PTT;
29. validate TF sequence numbers before storing fragments;
30. bound the fragment count to eight;
31. time out incomplete reassembly;
32. avoid feeding remote events back into local SUBSPACE transmission;
33. tolerate packet loss without blocking unrelated traffic.

A conforming sender should:

34. keep lines within the configured bearer budget;
35. use printable ASCII;
36. replace or escape field delimiters;
37. bound callsigns;
38. bound event text;
39. rate-limit logical event transmission;
40. avoid requiring an application handshake.

23. Implementation Notes for Future Maintainers

The most important maintenance boundary is:

```
Field Log
|
| offerSubspace()
v
```

```

Subspace.kt
|
| send(text)
v
MeshLink.kt
|
| Meshtastic protobuf
v
Bluetooth GATT
|
v
Meshtastic node
|
v
LoRa mesh

```

Do not move field-log evidence into the SUBSPACE queue.

Do not add reliability assumptions to `MeshLink`.

Do not make `Subspace.pump()` wait for a radio acknowledgement that does not exist.

Do not increase event rate merely because a phone can generate detections faster than the radio can safely carry them.

If a future protocol version adds reliable delivery, it should define an explicit new frame type and state machine rather than silently changing `TF` semantics.

24. Known 0.6.138 Limitations

The following are known properties of the current implementation, not hypothetical concerns:

41. **TF** identifiers are sender-derived and only four hexadecimal characters are transmitted.
42. Two simultaneous multi-frame events from the same sender can collide in the receiver reassembly map.
43. A timed-out event is spoken as a partial concatenation without an explicit “incomplete” marker.
44. `‘AH’` and `‘POS’` duplicate position information.
45. `PTT` exists in both compact and coordinate-bearing forms.
46. `‘TC’` is recognized by the receive path but is not generated by the 0.6.138 source tree.
47. The receiver's ALOHA timestamp is currently not used as a freshness value.
48. The `‘R’` ALOHA reply mechanism exists in code but is disabled.
49. The SUBSPACE rate limit counts logical events, while each fragmented event consumes multiple bearer transmissions.
50. The bearer default destination is broadcast, although a configurable destination exists.
51. The embedded repository README describes an older software status than the 0.6.138 source artifact being specified. Maintainers should treat the actual source tree/version tag as authoritative when reconciling documentation.

These limitations are documented explicitly so that a future maintainer does not mistake them for accidental omissions in this RFC.

25. Recommended Revision: SUBSPACE/1

A future revision should preserve the basic philosophy while correcting the collision and identity problems.

A candidate revision should introduce:

```
TF1|sender|message-id|i/n|category|body
```

where:

- `sender` is stable across all frame types;
- `message-id` is unique per logical event;
- `i/n` remains explicit;
- `category` remains human-readable;
- `body` remains bounded.

The sender identity should preferably be derived from a stable application identity rather than independently chosen callsign, radio-tail, and GUID fragments.

The revision should also consider:

- explicit `INCOMPLETE` indication for timeout delivery;
- one canonical discovery frame instead of both `AH` and `POS`;
- explicit protocol versioning;
- optional authentication;
- a bounded replay/duplicate cache;
- per-message rather than per-sender fragment identity;
- an explicit privacy/location policy;
- a defined maximum text length independent of the underlying Meshtastic preset.

Importantly, these are **future protocol recommendations**, not claims about 0.6.138.

26. IANA Considerations

This document has no IANA actions.

The protocol is an application convention carried inside Meshtastic text messages and does not request an Internet protocol number, URI scheme, media type, or port assignment.

27. Relationship to Other Protocols

SUBSPACE deliberately borrows concepts rather than claiming protocol compatibility.

27.1 ALOHA

The AH concept resembles an unsolicited presence transmission. SUBSPACE does not implement the ALOHA random-access algorithm.

27.2 UDP

PTT is datagram-like: one text message represents one application utterance without a connection or acknowledgement. SUBSPACE is not UDP and does not use an IP transport.

27.3 IP fragmentation

TF borrows the general idea of identifying fragments and numbering them. It is not compatible with IPv4 fragmentation and does not reproduce IPv4 semantics.

27.4 APRS

AH and POS carry an identity, location, and beacon-like presence model reminiscent of APRS. SUBSPACE does not implement APRS or AX.25.

27.5 Neighbor discovery

Periodic peer announcements resemble neighbor-discovery HELLO traffic. SUBSPACE does not implement IPv6 Neighbor Discovery or OLSR.

27.6 TCP

SUBSPACE deliberately omits TCP-like connection establishment, acknowledgement, sequencing, retransmission, and stream semantics.

28. References

Normative / protocol references

[RFC2119] S. Bradner, “Key words for use in RFCs to Indicate Requirement Levels,” RFC 2119, March 1997.

[RFC8174] B. Leiba, “Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words,” RFC 8174, May 2017.

[RFC768] J. Postel, “User Datagram Protocol,” RFC 768, August 1980.

[RFC791] J. Postel, “Internet Protocol,” RFC 791, September 1981.

[RFC4861] T. Narten, E. Nordmark, W. Simpson, and H. Soliman, “Neighbor Discovery for IP version 6 (IPv6),” RFC 4861, September 2007.

[RFC3626] T. Clausen and P. Jacquet, “Optimized Link State Routing Protocol (OLSR),” RFC 3626, October 2003.

Informative references

[Meshtastic] Meshtastic documentation, project introduction and device documentation.

[Tricorder-Source] Project Tricorder source tree, build 0.6.138, especially `Subspace.kt`, `MeshLink.kt`, `MeshProto.kt`, and the corresponding Comlink handling in `MainActivity.kt`.

29. Appendix A — Wire Examples

A.1 Discovery

```
AH|KIRK|7A3F|41.50321|-74.01022|09:42
POS|KIRK|41.50321|-74.01022
```

A.2 Compact PTT

```
PTT|SPOCK|Sensor report ready
```

A.3 Coordinate-bearing PTT

PTT|SPOCK|41.50321|-74.01022|Sensor report ready

A.4 Beacon-shaped PTT

PTT|SPOCK|41.50321|-74.01022|aloha

A.5 Single-frame event

TF|A91C|1/1|flight|09:42 41.503,-74.010 aircraft detected

A.6 Multi-frame event

TF|A91C|1/2|civic|09:43 41.503,-74.010 civic alert: road closure...

TF|A91C|2/2|civic|northbound near the bridge

30. Appendix B — Reference Implementation Constants

Parameter	0.6.138 behavior
`TF` chunk size	140 characters
Maximum `TF` fragments	8
Normal event label cap	480 characters
Event rate	1–5 logical events/minute
Event default enabled	weapon, civic, flight
Discovery first attempt	8 seconds
Discovery interval	1 or 3 minutes
Discovery retry without GPS/link	20 seconds
Bearer drip default	2500 ms
Bearer drip range	800–8000 ms
PTT growth window	8 seconds
TF reassembly timeout	2.5 seconds
SUBSPACE sidecar cap	400 records
Default bearer destination	`ffffff`
Meshtastic hop limit in encoder	3
Radio text application	`TEXT MESSAGE APP`

31. Appendix C — Design Principle

The protocol can be reduced to one sentence:

The radio carries bytes; SUBSPACE gives those bytes instrument meaning.

That separation is the reason the Tricorder can replace the radio bearer later without redesigning the sensing system, and why the radio can continue to evolve without becoming responsible for Tricorder semantics.

The protocol is therefore intentionally small at the wire and explicit in the application.

It is not trying to make LoRa behave like Ethernet.

It is trying to make two independent instruments understand one another.